

Unit-1 (2 marks)

1. With the syntax write the purpose of ordinal() method.

Syntax: `final int ordinal ()`: obtain a value that indicates an enumeration constant's position in the list of constants. This is called its ordinal value

2. List two key characteristics of enumeration constants in Java.

- In Java, an enumeration defines a **class type**- By making enumerations into classes, the capabilities of the enumeration are greatly expanded. For example, in Java, an enumeration can have **constructors, methods, and instance variables**
- Each enumeration constant is **public, static and final** by default

3. What is an enumeration? How enumeration can be created?

- ♦ An **enumeration** is a list of named constants.
- ♦ An enumeration is created using the **enum** keyword.
- ♦ For example, here is a simple enumeration that lists various apple varieties:

// An enumeration of apple varieties.

```
enum Apple { Jonathan, GoldenDel, RedDel, Winesap, Cortland }
```

- ♦ The identifiers Jonathan, GoldenDel, and so on, are called enumeration constants.

4. Differentiate values() and valueOf() methods in Java enumerations.

- ♦ All enumerations automatically contain two predefined methods: `values()` and `valueOf()`.
- ♦ Their general forms are shown here:

```
public static enum-type [ ] values( )
```

```
public static enum-type valueOf(String str )
```

- ♦ The **values()** method returns an array that contains a list of the enumeration constants
- ♦ The **valueOf()** method returns the enumeration constant whose value corresponds to the string passed in str.

- ◆ In both cases, **enum-type** is the type of the enumeration. For example, in the case of the Apple enumeration, the return type of `Apple.valueOf("Winesap")` is `Winesap`.

5. Provide an example of how to use the `values()` method to iterate through all the constants in an enumeration.

The following program demonstrates the `values()` and `valueOf()` methods:

```
enum Apple {
    Jonathan, GoldenDel, RedDel, Winesap, Cortland }
class EnumDemo2 {
    public static void main(String args[]) {
        Apple ap; System.out.println("Here are all Apple
        constants:");
        //use values()
        Apple allapples[] = Apple.values();
        for(Apple a : allapples)
            System.out.println(a);
        System.out.println();
        //use valueOf()
        ap = Apple.valueOf("Winesap");
        System.out.println("ap contains " + ap);
    }
}
```

output
 Here are all Apple
 constants:
 Jonathan
 GoldenDel
 RedDel
 Winesap
 Cortland
 ap contains Winesap

6. Give the functionalities of `compareTo()`, and `equals()` methods in Java enumerations.

- ◆ **boolean equals(Object other)** returns true if the specified object is equal to this enum constant
- ◆ **int compareTo(E o)** compares this enum with the specified object for order; returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object

7. Why are type wrappers used in Java?

- ◆ Type wrappers convert primitive data types into objects.
- ◆ Objects are needed if we wish to modify the arguments passed into a method.
- ◆ The classes in **java.util** package handles only objects and hence wrapper classes help in this case also.

- ◆ Data structures in the Collection framework, such as **ArrayList** and **Vector**, store only objects (reference types) and not primitive types.
- ◆ An object is needed to support synchronization in multithreading
- ◆ They relate directly to Java's **autoboxing** feature

8. What is the purpose of the `doubleValue()` method in a numeric wrapper class?(similarly other methods)

`doubleValue()` returns the value of an object as a double, **`floatValue()`** returns the value as a float, and so on. These methods are implemented by each of the numeric type wrappers.

9. What is the benefit of autoboxing and auto-unboxing in Java?

- ◆ *The process of encapsulating a value within an object is called boxing.* Thus, in the program, this line boxes the value 100 into an Integer: **`Integer iOb = new Integer(100);`** The process of extracting a value from a type wrapper is called *unboxing*. For example, the program unboxes the value in iOb with this statement: **`int i = iOb.intValue();`**

10. What is retention policy? How to set retention policy? Give an example

- ◆ A **retention policy** determines at what point an annotation is discarded.
- ◆ Java defines three such policies, which are encapsulated within the **`java.lang.annotation.RetentionPolicy` enumeration**. They are SOURCE, CLASS, and RUNTIME
- ◆ MyAnno uses **`@Retention`** to specify the RUNTIME retention policy. Thus, MyAnno will be available to the JVM during program execution.

```
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno
{ String str(); int val();
}
```

11. List any two retention policies with its purpose. Retention Policy:

- **SOURCE** is retained only in the source file and is discarded during compilation.
- **CLASS** is stored in **the .class file** during compilation. It is not available through the JVM during run time.
- **RUNTIME** is stored in **the .class file** during compilation and is available through the JVM during run time

Thus, RUNTIME retention offers the greatest annotation persistence.

- A retention policy is specified for an annotation by using one of Java's built-in annotations: **@Retention**.

12. What is the purpose of setting default values to annotation member? Write the general form for setting default values.

- You can give annotation members default values that will be used if no value is specified when the annotation is applied

type member() default value ;

- Here, value must be of a type compatible with type.

// An annotation type declaration that includes defaults.

@Retention(RetentionPolicy.RUNTIME)

@interface MyAnno {

String str() default "Testing";

int val() default 9000; }

This declaration gives a default value of "Testing" to str and 9000 to val

13. Define Marker Annotation and Single Member Annotation.

Marker Annotations: A marker annotation is a special kind of annotation that contains no members. Its sole purpose is to mark an item. Thus, its presence as an annotation is sufficient. The best way to determine if a marker annotation is present is to use the method **isAnnotationPresent()**, which is defined by the **AnnotatedElement** interface.

Single-Member Annotations: A single-member annotation contains only one member. It works like a normal annotation except that it allows a shorthand form of specifying the value of the member. When only one member is present, you can simply specify the value for that member when the annotation is applied—you don't need to specify the name of the member. However, in order to use this shorthand, the name of the member must be value

14. List any two built in annotations with its purpose.

@Retention- It specifies the retention policy
@Documented- is a marker interface that tells a tool that an annotation is to be documented

- The **@Target annotation** specifies the types of items to which an annotation can be applied **@Target** takes one argument, which is an array of constants of the **ElementType** enumeration. This argument specifies the types of declarations to which the annotation can be applied.

15. Write any two restrictions on annotation.

- ◆ First, no annotation can inherit another.
- ◆ Second, all methods declared by an annotation must be without parameters. Furthermore, they must return one of the following:
 - A primitive type, such as **int** or **double**
 - An object of type **String** or **Class**
 - An **enum** type
 - Another annotation type

16. What is the primary purpose of annotations in Java code?

- ◆ **Annotations, a** form of metadata, provide data about a program that is not part of the program itself. Annotations have no direct effect on the operation of the code they annotate.
- ◆ Annotations have a number of uses, among them:
 - **Information for the compiler** — Annotations can be used by the compiler to detect errors or suppress warnings.

- **Compile-time and deployment-time processing** — Software tools can process annotation information to generate code, XML files, and so forth.
- **Runtime processing** — Some annotations are available to be examined at runtime. —

17. How are annotations declared in Java? Give example Declaration Syntax (for declaring the annotation)

```
@ interface AnnotationName{  
    dataType member_name ( ) [( @default value)]  
}
```

Utilization Syntax (we can utilize annotations for variables, methods , class, ...(i.e. almost all programming elements)

```
@AnnotationName(member1= value, memmebr2 =  
value2, ... memberN = valueN) programming_element
```

For example: `@MyAnno(age=20,name="Ram ",country="Nepal")`

18. How can you retrieve all annotations with the RUNTIME retention policy associated with an element in Java reflection? Give its syntax

You can obtain all annotations that have RUNTIME retention that are associated with an item by calling `getAnnotations( )` on that item. `Annotation[ ] getAnnotations( )`

- It returns an array of the annotations.
- `getAnnotations( )` can be called on objects of type Class, Method, Constructor, and Field, among others

19. Name any two commonly used built-in annotations and briefly describe their purpose?

Java defines many built-in annotations. Most are specialized, but nine are general purpose. Of these, four are imported from `java.lang.annotation`:

- `@Retention`
- `@Documented`,
- `@Target`, and `@Inherited`.
- Five—`@Override`, `@Deprecated`, `@FunctionalInterface`, `@SafeVarargs`, and `@SuppressWarnings`

- `@Retention`- It specifies the retention policy
- `@Documented`- is a marker interface that tells a tool that an

annotation is to be documented

- The **@Target annotation** specifies the types of items to which an annotation can be applied. **@Target** takes one argument, which is an array of constants of the **ElementType** enumeration. This argument specifies the types of declarations to which the annotation can be applied.

20. What are some key characteristics of a Java Bean?

- ◆ A Java Bean is a software component that has been designed to be reusable in a variety of different environments.
- ◆ There is no restriction on the capability of a Bean.
- ◆ It may perform a simple function, such as obtaining an inventory value, or a complex function, such as forecasting the performance of a stock portfolio.
- ◆ A Bean may be visible to an end user.
- ◆ One example of this is a button on a graphical user interface.
- ◆ A Bean may also be invisible to a user.
- ◆ Software to decode a stream of multimedia information in real time is an example of this type of building block

21. List two advantages of Java Beans

- ◆ A Bean obtains all the benefits of Java's "write-once, run-anywhere" paradigm.
- ◆ The properties, events, and methods of a Bean that are exposed to another application can be controlled.
- ◆ Auxiliary software can be provided to help configure a Bean.

22. Why is introspection essential for Java Beans technology?

- ◆ At the core of Java Beans is introspection.
- ◆ This is the process of analyzing a Bean to determine its capabilities.
- ◆ This is an essential feature of the Java Beans API because it allows another application, such as a design tool, to obtain information about a component.
- ◆ Without introspection, the Java Beans technology could not operate

23. What is the difference between a simple property and an indexed property?

Simple Properties: A simple property has a single value. It can be identified by the following design patterns, where N is the name of the property and T is its type:

public T getN()

public void setN(T arg)

- A read/write property has both of these methods to access its values.
- A read-only property has only a get method.
- A write-only property has only a set method.

Indexed Properties: An indexed property consists of multiple values. It can be identified by the following design patterns, where N is the name of the property and T is its type:

public T getN(int index);

public void setN(int index, T value);

public T[] getN();

public void setN(T values[]);

Here is an indexed property called data along with its getter and setter methods

24. What are the two main components used to define a simple property in a Java Bean. Briefly explain their roles

Refer the above answer of simple property

25. What are the key differences between bound properties and constrained properties in Java Beans?

Bound and Constrained Properties

- ◆ A Bean that has a *bound* property generates an event when the property is changed.
- ◆ The event is of type **PropertyChangeEvent** and is sent to objects that previously registered an interest in receiving such notifications.
- ◆ A class that handles this event must implement the **PropertyChangeListener** interface
- ◆ A Bean that has a **constrained property** generates an event when an attempt is made to change its value. It also generates an event of type **PropertyChangeEvent**.
- ◆ It too is sent to objects that previously registered an interest in receiving such notifications

26. What is persistence in JavaBeans?

Persistence: Persistence is the ability to save the current state of a Bean, including the values of a Bean's properties and instance variables, to nonvolatile storage and to retrieve them at a later time.

27. What are customizers?

Customizers

- ◆ A Bean developer can provide a customizer that helps another developer configure the Bean.
- ◆ A customizer can provide a step-by-step guide through the process that must be followed to use the component in a specific context.
- ◆ Online documentation can also be provided.
- ◆ A Bean developer has great flexibility to develop a customizer that can differentiate his or her product in the marketplace

(4 to 6 marks)

1. With an example explain how enumeration values are used to control a switch statement.

An enumeration value can also be used to control a switch statement. Of course, all of the case statements must use constants from the same enum as that used by the switch expression. For example, this switch is perfectly valid:

```
// Use an enum to control a switch statement.
```

```
switch(ap) {
```

```
case Jonathan:
```

```
// ...
```

```
case Winesap:
```

```
// ...
```

The following program puts together all of the pieces and demonstrates the **Apple** enumeration:

```
// An enumeration of apple varieties.
```

```
enum Apple {
```

```
Jonathan, GoldenDel, RedDel, Winesap, Cortland }
```

```
class EnumDemo {
```

```
public static void main(String args[]) {
```

```
Apple ap; ap = Apple.RedDel;
```

```
// Output an enum value.
```

```
System.out.println("Value of ap: " + ap);
```

```
System.out.println();
```

```

ap = Apple.GoldenDel;
// Compare two enum values.
if(ap == Apple.GoldenDel)
System.out.println("ap contains GoldenDel.\n");
// Compare two enum values.
if(ap == Apple.GoldenDel)
System.out.println("ap contains GoldenDel.\n");
// Use an enum to control a switch statement.
switch(ap) {
case Jonathan: System.out.println("Jonathan is red."); break;
case GoldenDel: System.out.println("Golden Delicious is
yellow."); break;
case RedDel: System.out.println("Red Delicious is red.");
break;
case Winesap: System.out.println("Winesap is red.");
break;
case Cortland: System.out.println("Cortland is red.");
break;
}
}
} output: Value of ap: RedDel
ap contains GoldenDel. Golden Delicious is yellow.

```

2.Demonstrate the usage of the valueOf() and values() methods with an example

- ◆ All enumerations automatically contain two predefined methods: values() and valueOf().
- ◆ Their general forms are shown here:

public static enum-type [] values()

public static enum-type valueOf(String str)

- ◆ The values() method returns an array that contains a list of the enumeration constants
 - ◆ The valueOf() method returns the enumeration constant whose value corresponds to the string passed in str.
 - ◆ In both cases, enum-type is the type of the enumeration.
- The following program demonstrates the values() and valueOf() methods:

```

// Use the built-in enumeration methods. An enumeration of
apple varieties.
enum Apple {
Jonathan, GoldenDel, RedDel, Winesap, Cortland }
class EnumDemo2 {

```

```

public static void main(String args[]) {
    Apple ap;
    System.out.println("Here are all Apple constants:");
    //use values()
    Apple allapples[] = Apple.values();
    for(Apple a : allapples)
        System.out.println(a);
    System.out.println();
    //use valueOf()
    ap = Apple.valueOf("Winesap");
    System.out.println("ap contains " + ap);
}
}

```

Here are all Apple constants: Jonathan GoldenDel RedDel Winesap Cortland ap contains Winesap
--

3.What does ordinal(), compareTo() and equals() method do in Enum? Give an example.

- ◆ **boolean equals(Object other)** returns true if the specified object is equal to this enum constant
- ◆ **int compareTo(E o)** compares this enum with the specified object for order; returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object
- ◆ **final int ordinal()** obtain a value that indicates an enumeration constant's position in the list of constants. This is called its **ordinal value**

```

// Demonstrate ordinal(), compareTo(), and equals().
// An enumeration of apple varieties.
enum Apple { Jonathan, GoldenDel, RedDel, Winesap, Cortland }
class EnumDemo4 {
    public static void main(String args[]) {
        Apple ap, ap2, ap3;
        // Obtain all ordinal values using ordinal().
        System.out.println("Here are all apple constants" + " and their ordinal values: ");
        for(Apple a : Apple.values())
            System.out.println(a + " " + a.ordinal());
        ap = Apple.RedDel; ap2 = Apple.GoldenDel;
        ap3 = Apple.RedDel;
        System.out.println();
        // Demonstrate compareTo() and equals()
        if(ap.compareTo(ap2) < 0)

```



```

System.out.println(ap + " comes before " + ap2);
if(ap.compareTo(ap2) > 0)
System.out.println(ap2 + " comes before " + ap);
if(ap.compareTo(ap3) == 0)
System.out.println(ap + " equals " + ap3);
System.out.println();
if(ap.equals(ap2))
System.out.println("Error!");
if(ap.equals(ap3))
System.out.println(ap + " equals " + ap3);
if(ap == ap3) System.out.println(ap + " == " + ap3);
} output:
Jonathan 0   GoldenDel 1   RedDel 2   Winesap 3 Cortland 4
GoldenDel comes before RedDel
RedDel equals RedDel
RedDel equals RedDel
RedDel == RedDel

```

4. Java Enumerations Are Class Types. Explain with an example.

- ◆ A Java enumeration is a class type.
- ◆ Although you don't instantiate an **enum** using **new**, it otherwise has much the same capabilities as other classes.
- ◆ **enum** defines a **class** gives powers to the *Java enumeration*
- ◆ For example, you can give them constructors, add instance variables and methods, and even implement interfaces.
- ◆ It is important to understand that each enumeration constant is an object of its enumeration type.
- ◆ Thus, when you define a constructor for an enum, the constructor is called when each enumeration constant is created.
- ◆ Also, each enumeration constant has its own copy of any instance variables defined by the enumeration.

// Use an enum constructor, instance variable, and method.

```

enum Apple { Jonathan(10), GoldenDel(9), RedDel(12),
Winesap(15), Cortland(8);
private int price; // price of each apple
// Constructor
Apple(int p) { price = p; }
int getPrice() { return price; }
}

```

```

class EnumDemo3 {
    public static void main(String args[]) {
        Apple ap;
        // Display price of Winesap.
        System.out.println("Winesap costs " + Apple.Winesap.getPrice()
            + " cents.\n");
        // Display all apples and prices.
        System.out.println("All apple prices:");
        for(Apple a : Apple.values())
            System.out.println(a + " costs " + a.getPrice() + " cents.");
    }
}

```

output : Winesap costs 15 cents.
 All apple prices: Jonathan costs 10 cents.
 GoldenDel costs 9 cents. RedDel costs 12 cents.
 Winesap costs 15 cents. Cortland costs 8 cents

5.Explain with example how Enumerations can Inherit Enum?

Enumerations Inherit Enum:

- ◆ All enumerations automatically inherit one class:
java.lang.Enum.
 - ◆ **String toString()** returns the name of this enum constant, as contained in the declaration
 - ◆ **boolean equals(Object other)** returns true if the specified object is equal to this enum constant
 - ◆ **int compareTo(E o)** compares this enum with the specified object for order; returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object
 - ◆ **static enum-type valueOf(String s)** returns the enumerated object whose name is s
 - ◆ **static enum-type[] values()** returns an array of the enumeration objects
 - ◆ **final int ordinal()** obtain a value that indicates an enumeration constant's position in the list of constants. This is called its **ordinal value**
- ```

// Demonstrate ordinal(), compareTo(), and equals().
// An enumeration of apple varieties.
enum Apple { Jonathan, GoldenDel, RedDel, Winesap, Cortland }
class EnumDemo4 { public static void main(String args[]) {

```

```

 Apple ap, ap2, ap3;
 // Obtain all ordinal values using ordinal().
 System.out.println("Here are all apple constants" + " and
their ordinal values: ");
 for(Apple a : Apple.values())
 System.out.println(a + " " + a.ordinal());
 ap = Apple.RedDel; ap2 = Apple.GoldenDel;
 ap3 = Apple.RedDel;
 System.out.println();
 // Demonstrate compareTo() and equals()
 if(ap.compareTo(ap2) < 0)
 System.out.println(ap + " comes before " + ap2);
 if(ap.compareTo(ap2) > 0)
 System.out.println(ap2 + " comes before " + ap);
 if(ap.compareTo(ap3) == 0)
 System.out.println(ap + " equals " + ap3);
 System.out.println(); if(ap.equals(ap2))
 System.out.println("Error!");
 if(ap.equals(ap3)) System.out.println(ap + " equals " + ap3);
 if(ap == ap3) System.out.println(ap + " == " + ap3);
} output:
Jonathan 0 GoldenDel 1 RedDel 2 Winesap 3 Cortland 4
GoldenDel comes before RedDel
RedDel equals RedDel
RedDel equals RedDel
RedDel == RedDel

```

## 6. Describe the Wrapper classes available for primitive types.

- ◆ **Wrapper class** in java provides the mechanism to convert primitive into object and object into primitive
- ◆ For example, the Integer class contains a method to convert an integer stored in a String to an int value:

```

int num;
num = Integer.parseInt(str);
Or
float fltnum;
fltnum = Float.parseFloat(str);

```

- ◆ **Character:** **Character** is a wrapper around a **char**. The constructor for Character is: **Character(char ch)**



Here, **ch** specifies the character that will be wrapped by the **Character object** being created.

To obtain the char value contained in a **Character object**, call **charValue( )**, general form of this method is shown here: **char charValue( )** It returns the encapsulated character

◆ **Boolean:** **Boolean** is a wrapper around **boolean** values. It defines these constructors: **Boolean(boolean boolValue)**

**Boolean(String boolString)**

In the first version, **boolValue** must be either **true** or **false**.

In the second version, if **boolString** contains the string "**true**" (in uppercase or lowercase), then the new **Boolean** object will be true. Otherwise, it will be false. To obtain a **boolean** value from a **Boolean object**, use **booleanValue( )**, general form of this method is shown here: **boolean booleanValue( )** It returns the **boolean** equivalent of the invoking object.

#### ◆ The Numeric Type Wrappers

These are **Byte**, **Short**, **Integer**, **Long**, **Float**, and **Double**. All of the numeric type wrappers inherit the abstract class **Number**. **Number** declares methods that return the value of an object in each of the different number formats. These methods are shown here:

**byte byteValue( )**

**double doubleValue( )**

**float floatValue( )**

**int intValue( )**

**long longValue( )**

**short shortValue( )**

For example, **doubleValue( )** returns the value of an object as a **double**, **floatValue( )** returns the value as a float, and so on. These methods are implemented by each of the numeric type wrappers.

For example, here are the constructors defined for **Integer**:

**Integer(int num)**

**Integer(String str)**

If **str** does not contain a valid numeric value, then a **NumberFormatException** is thrown.

// Demonstrate a type wrapper.

class Wrap {

public static void main(String args[]) {

```

Integer iOb = new Integer(100);
int i = iOb.intValue();
System.out.println(i + " " + iOb); // displays 100 100
}

```

### 7. What Is Autoboxing And Unboxing? Explain with an example.

**Autoboxing:** Automatic conversion of primitive types to the object of their corresponding wrapper classes is known as autoboxing.

**Auto-unboxing** is the process by which the value of a boxed object is automatically extracted (unboxed) from a type wrapper when its value is needed. There is no need to call a method such as `intValue()` or `doubleValue()`. For example, here is the modern way to construct an Integer object that has the value 100:

```

Integer iOb = 100; // autobox an int
unbox an object, simply assign that object reference to a
primitive-type variable. For example, to unbox iOb, you can
use this line: int i = iOb; // auto-unbox
// Demonstrate autoboxing / unboxing.
class AutoBox {
 public static void main(String args[]) {
 Integer iOb = 100; // autobox an int
 int i = iOb; // auto-unbox
 System.out.println(i + " " + iOb); // displays 100 100
 }
}

```

### 8. With an example explain the steps involved to obtain annotation at run time using reflection.

#### Steps to use Reflection:

- The first step to using reflection is to obtain a `Class` object that represents the class whose annotations you want to obtain. `Class` is one of Java's built-in classes and is defined in **java.lang**.
- One of the easiest is to call **getClass()**, which is a method defined by `Object` final **Class<?> getClass()**. It returns the `Class` object that represents the invoking object **<?>** - This is related to Java's generics feature.
- If you want to obtain the annotations associated with a

specific item declared within a class.

- First obtain an object that represents that item.
- For example, Class supplies (among others) the

**getMethod()** - return objects of type Method

**getField()** - return objects of type Field

**getConstructor()** - return objects of type Constructor

**getAnnotation()**- From a Class, Method, Field, or Constructor object, you can obtain a specific annotation associated with that object by calling **getAnnotation()**.

General form : **<A extends Annotation>**

**getAnnotation(Class<A> annoType)**

**annoType** is a Class object that represents the annotation in which you are interested. The method returns a reference to the annotation.

```
import java.lang.annotation.*;
```

```
import java.lang.reflect.*;
```

```
// An annotation type declaration.
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@interface MyAnno {
```

```
String str();
```

```
int val();
```

```
}
```

```
class Meta {
```

```
// Annotate a method.
```

```
@MyAnno(str = "Annotation Example", val = 100)
```

```
public static void myMeth() {
```

```
Meta ob = new Meta();
```

```
// Obtain the annotation for this method
```

```
// and display the values of the members.
```

```
try {
```

```
// First, get a Class object that represents this class.
```

```
Class c = ob.getClass();
```

```
// Now, get a Method object that represents this method.
```

```
Method m = c.getMethod("myMeth");
```

```
// Next, get the annotation for this class.
```

```
MyAnno anno = m.getAnnotation(MyAnno.class);
```

```
// Finally, display the values.
```

```
System.out.println(anno.str() + " " + anno.val());
```



```

 } catch (NoSuchMethodException exc) {
 System.out.println("Method Not Found.");
 }
}

public static void main(String args[]) {
 myMeth();
}
}

```

Output: Annotation Example 100

### 9. What Are Annotations? What are the three retention policies defined by the RetentionPolicy enumeration in Java?

◆ **Annotations**, a form of metadata, provide data about a program that is not part of the program itself. Annotations have no direct effect on the operation of the code they annotate.

◆ Annotations have a number of uses, among them:

• **Information for the compiler** — Annotations can be used by the compiler to detect errors or suppress warnings.

• **Compile-time and deployment-time processing** — Software tools can process annotation information to generate code, XML files, and so forth.

• **Runtime processing** — Some annotations are available to be examined at runtime.

#### Specifying a Retention Policy:

- A retention policy determines at what point an annotation is discarded.

- Java defines three such policies, which are encapsulated within the **java.lang.annotation.RetentionPolicy enumeration**

- They are **SOURCE**, **CLASS**, and **RUNTIME**

◆ **SOURCE** is retained only in the source file and is discarded during compilation.

- **CLASS** is stored in the *.class file* during compilation. It is not available through the JVM during run time.

- **RUNTIME** is stored in the *.class file* during compilation and is available through the JVM during run time

◆ Thus, **RUNTIME** retention offers the greatest annotation persistence.

- A retention policy is specified for an annotation by using one of Java's built-in annotations: **@Retention**.

**For example:**

```
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno
{ String str(); int val();
}
```

## 10. Write an example program to illustrate reflection.

```
import java.lang.annotation.*;
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno {
String str();
int val();
}
class Meta {
// myMeth now has two arguments.
@MyAnno(str = "Two Parameters", val = 19)
public static void myMeth(String str, int i)
{
Meta ob = new Meta();
try {
Class c = ob.getClass();
// Here, the parameter types are specified.
Method m = c.getMethod("myMeth", String.class, int.class);
MyAnno anno = m.getAnnotation(MyAnno.class);
System.out.println(anno.str() + " " + anno.val());
} catch (NoSuchMethodException exc) {
System.out.println("Method Not Found.");
}
}
public static void main(String args[]) {
myMeth("test", 10);
}
}
Output:Two Parameters 19
```

myMeth( ) takes a String and an int parameter.

To obtain information about this method, getMethod( ) must be called as shown here:

```
Method m = c.getMethod("myMeth", String.class, int.class);
```

Here, the Class objects representing String and int are passed as additional arguments.

### 11. Explain any four Built-in annotations.

Java defines many built-in annotations. Most are specialized, but nine are general purpose.

- Of these, four are imported from **java.lang.annotation**:

- **@Retention**
- **@Documented**.
- **@Target**, and **@Inherited**.

- Five—**@Override**, **@Deprecated**, **@FunctionalInterface**, **@SafeVarargs**, and

- @SuppressWarnings**—are included in java.lang

- **@Retention** - It specifies the retention policy

- **@Documented** - is a marker interface that tells a tool that an annotation is to be documented

- The **@Target** annotation specifies the types of items to which an annotation can be applied

**@Target** takes one argument, which is an array of constants of the **ElementType** enumeration. This argument specifies the types of declarations to which the annotation can be applied.

For example, to specify that an annotation applies only to fields and local variables, you can use this **@Target** annotation:

```
@Target({
```

```
ElementType.FIELD,ElementType.LOCAL_VARIABLE })
```

- **@Inherited** - causes the annotation for a superclass to be inherited by a subclass

- **@Override** - annotation that can be used only on methods. A method annotated with **@Override** must override a method from a superclass. If it doesn't, a compile-time error will result.

- **@Deprecated** - It indicates that a declaration is obsolete and has been replaced

- **@FunctionalInterface** - marker annotation added by JDK 8 and designed for use on interfaces. It indicates that the annotated interface is a functional interface. A functional interface is an interface that contains one and only one abstract method. Functional interfaces are used by lambda expressions.

- **@SafeVarargs** - is a marker annotation that can be applied to



methods and constructors. It indicates that no unsafe actions related to a varargs parameter occur

- **@SuppressWarnings** - It specifies that one or more warnings that might be issued by the compiler are to be suppressed. The warnings to suppress are specified by name, in string form. newer form

## 12. How do you specify a default value for an annotation member? Explain with suitable example

### Using Default Values:

- You can give annotation members default values that will be used if no value is specified when the annotation is applied

#### **type member( ) default value ;**

- Here, value must be of a type compatible with type.

```
// An annotation type declaration that includes defaults.
@Retention(RetentionPolicy.RUNTIME)
```

```
@interface MyAnno {
 String str() default "Testing";
 int val() default 9000; }
This declaration gives a default value of "Testing" to str and 9000 to val. This means that neither value needs to be specified when @MyAnno is used. However, either or both can be given values if desired. Therefore, following are the four ways that @MyAnno can be used:
```

```
@MyAnno() // both str and val default
@MyAnno(str = "some string") // val defaults
@MyAnno(val = 100) // str defaults
@MyAnno(str = "Testing", val = 100) // no defaults
```

The following program demonstrates the use of default values in an annotation.

```
import java.lang.annotation.*;
import java.lang.reflect.*;
// An annotation type declaration that includes defaults.
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnno {
 String str() default "Testing";
 int val() default 9000; }
class Meta3 {
 // Annotate a method using the default values.
```

```

 @MyAnno()
 public static void myMeth()
 { Meta3 ob = new Meta3(); // Obtain the annotation for
 this method
 // and display the values of the members.
 try {
 Class c = ob.getClass();
 Method m = c.getMethod("myMeth")
 MyAnno anno = m.getAnnotation(MyAnno.class);
 System.out.println(anno.str() + " " + anno.val());
 }
 catch (NoSuchMethodException exc)
 {
 System.out.println("Method Not Found.");
 }
 }
 public static void main(String args[]) {
 myMeth();
 }
 }
} The output is shown here: Testing 9000

```

### 13. Write an example of defining a marker annotation

**Marker Annotations:** A marker annotation is a special kind of annotation that contains no members.

- Its sole purpose is to mark an item.
- Thus, its presence as an annotation is sufficient.
- The best way to determine if a marker annotation is present is to use the method *isAnnotationPresent()*, which is defined by the *AnnotatedElement* interface.
- Because a marker interface contains no members, simply determining whether it is present or absent is sufficient.

```

import java.lang.annotation.*;
import java.lang.reflect.*;
// A marker annotation.
@Retention(RetentionPolicy.RUNTIME)
@interface MyMarker { }
class Marker { // Annotate a method using a marker.
 // Notice that no () is needed.
 @MyMarker public static void myMeth() {
 Marker ob = new Marker();
 try { Method m = ob.getClass().getMethod("myMeth");

```

```
// Determine if the annotation is present.
if(m.isAnnotationPresent(MyMarker.class))
 System.out.println("MyMarker is present.");
}
catch (NoSuchMethodException exc)
{
 System.out.println("Method Not Found.");
}
} public static void main(String args[]) { myMeth(); }
```

} output, confirms that @MyMarker is present: **MyMarker is present**

#### 14. Write an example of defining a single-member annotation

**Single-Member Annotations:** A single-member annotation contains only one member. It works like a normal annotation except that it allows a shorthand form of specifying the value of the member.

- When only one member is present, you can simply specify the value for that member when the annotation is applied—you don't need to specify the name of the member.
- However, in order to use this shorthand, the name of the member must be value

```
import java.lang.annotation.*;
import java.lang.reflect.*;
// A single-member annotation.
@Retention(RetentionPolicy.RUNTIME)
@interface MySingle {
 int value(); // this variable name must be value
}
class Single {
 // Annotate a method using a single-member annotation.
 @MySingle(100)
 public static void myMeth() {
 Single ob = new Single();
 try {
 Method m = ob.getClass().getMethod("myMeth");
 MySingle anno = m.getAnnotation(MySingle.class);
 System.out.println(anno.value()); // displays 100
 } catch (NoSuchMethodException exc) {
```



```
System.out.println("Method Not Found.");
```

```
{
 public static void main(String args[]) { myMeth();
}
```

**Output: this program displays the value 100.** In the program, @MySingle is used to annotate myMeth( ), as shown here: @MySingle(100)

### **15. How Can You Retrieve all Annotations that have RUNTIME retention by use of reflection? Explain with an example**

#### **Obtaining All Annotations:**

- You can obtain all annotations that have RUNTIME retention that are associated with an item by calling **getAnnotations( )** on that item.

Annotation[] getAnnotations( )- It returns an array of the annotations.

- getAnnotations( ) can be called on objects of type Class, Method, Constructor, and Field, among others

// Show all annotations for a class and a method.

```
import java.lang.annotation.*;
```

```
import java.lang.reflect.*;
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@interface MyAnno {
```

```
 String str();
```

```
 int val();
```

```
}
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@interface What {
```

```
 String description();
```

```
}
```

```
@What(description = "An annotation test class")
```

```
@MyAnno(str = "Meta2", val = 99)
```

```
class Meta2 {
```

```
 @What(description = "An annotation test method")
```

```
 @MyAnno(str = "Testing", val = 100)
```

```
 public static void myMeth() {
```

```
 Meta2 ob = new Meta2();
```

```
 try {
```

```
 Annotation annos[] = ob.getClass().getAnnotations();
```

```
// Display all annotations for Meta2.
System.out.println("All annotations for Meta2:");
for(Annotation a : annos)
System.out.println(a);
System.out.println();
// Display all annotations for myMeth.
Method m = ob.getClass().getMethod("myMeth");
annos = m.getAnnotations();
System.out.println("All annotations for myMeth:");
for(Annotation a : annos)
System.out.println(a);
} catch (NoSuchMethodException exc) {
System.out.println("Method Not Found.");
}
}

public static void main(String args[]) { myMeth(); }
```

**The output is shown here:**

```
All annotations for Meta2:
@What(description=An annotation test class)
@MyAnno(str=Meta2, val=99)
All annotations for myMeth:
@What(description=An annotation test method)
@MyAnno(str=Testing, val=100)
```

## **16. Discuss the key advantages that Java Beans provide to component developers**

### **Advantages of Java Beans**

The following list enumerates some of the benefits that Java Bean technology provides for a component developer:

- A Bean obtains all the benefits of Java's "write-once, run-anywhere" paradigm.
- The properties, events, and methods of a Bean that are exposed to another application can be controlled.
- Auxiliary software can be provided to help configure a Bean.
- This software is only needed when the design-time parameters for that component are being set. It does not need to be included in the run-time environment.
- The configuration settings of a Bean can be saved in persistent storage and restored at a later time.
- A Bean may register to receive events from other objects and can generate events that are sent to other objects.

**17. What are the different properties of a Java Bean?  
Explain with examples**

There are two types of properties: **simple and indexed**

**Simple Properties:** A simple property has a single value. It can be identified by the following design patterns, where N is the name of the property and T is its type:

***public T getN()***

***public void setN(T arg)***

- A read/write property has both of these methods to access its values.
- A read-only property has only a get method.
- A write-only property has only a set method.
- Here are three read/write simple properties along with their getter and setter methods

```
public double getDepth() { return depth; }
public void setDepth(double d) { depth = d; }
public double getHeight() { return height; }
public void setHeight(double h) { height = h; }
public double getWidth() { return width; }
public void setWidth(double w) { width = w; }
```

**Indexed Properties:** An indexed property consists of multiple values. It can be identified by the following design patterns, where N is the name of the property and T is its type:

***public T getN(int index);***

***public void setN(int index, T value);***

***public T[] getN();***

***public void setN(T values[]);***

Here is an indexed property called data along with its getter and setter methods

```
private double data[];
public double getData(int index) { return data[index]; }
public void setData(int index, double value) {
 data[index] = value;
}
public double[] getData() { return data; }
public void setData(double[] values) {
 data = new double[values.length];
 System.arraycopy(values, 0, data, 0, values.length); }
```

**18. Write short note on the following**



**i. Bound and Constrained properties**

**ii. Persistence**

**Bound and Constrained properties:** A Bean that has a *bound* property generates an event when the property is changed.

- The event is of type **PropertyChangeEvent** and is sent to objects that previously registered an interest in receiving such notifications.
- A class that handles this event must implement the **PropertyChangeListener** interface
- A Bean that has a **constrained property** generates an event when an attempt is made to change its value. It also generates an event of type **PropertyChangeEvent**.
- It too is sent to objects that previously registered an interest in receiving such notifications.
- However, those other objects have the ability to veto the proposed change by throwing a **PropertyVetoException**.
- This capability allows a Bean to operate differently according to its run-time environment.
- A class that handles this event must implement the **VetoableChangeListener** interface

**19. Write a note on simple properties in java Beans.**

*Refer 17 answer in long answer*

**20. Write a note on indexed properties in java Beans**

*Refer 17 answer in long answer*

**21. What are the steps to be followed in creating a new Bean?**

four are of particular interest: **Introspector**  
**PropertyDescriptor**, **EventSetDescriptor**, and  
**MethodDescriptor**

**A Bean Example:**

illustrates various aspects of Bean programming, including introspection and using a BeanInfo class. It also makes use of the Introspector, PropertyDescriptor, and EventSetDescriptor classes. The example uses three classes. The first is a Bean called Colors, shown here:

```
import java.awt.*;
import java.awt.event.*;
```

```

import java.io.Serializable;
public class Colors extends Canvas implements Serializable
{ transient private Color color; // not persistent
 private boolean rectangular; // is persistent
 public Colors() {
 addMouseListener(new MouseAdapter() {
 public void mousePressed(MouseEvent me) {
 change();
 }
 });

 rectangular = false; setSize(200, 100); change();
 }

 public boolean getRectangular() {
 return rectangular;
 }

 public void setRectangular(boolean flag) {
 this.rectangular = flag; repaint();
 }

 public void change() {
 color = randomColor(); repaint();
 }

 private Color randomColor() {
 int r = (int)(255*Math.random());
 int g = (int)(255*Math.random());
 int b = (int)(255*Math.random());
 return new Color(r, g, b);
 }

 public void paint(Graphics g) { Dimension d = getSize();
 int h = d.height; int w = d.width; g.setColor(color);
 if(rectangular) g.fillRect(0, 0, w-1, h-1);
 else g.fillOval(0, 0, w-1, h-1);
 }
}

```

#### // A Bean information class.

```

import java.beans.*;
public class ColorsBeanInfo extends SimpleBeanInfo
{ public PropertyDescriptor[] getPropertyDescriptors()
{
 try {
 PropertyDescriptor rectangular = new
 PropertyDescriptor("rectangular", Colors.class);
 PropertyDescriptor pd[] = {rectangular};
 return pd;
 }
 catch(Exception e)
 { System.out.println("Exception caught. " + e);
 }
 return null;
}
}

```

- The final class is called `IntrospectorDemo`. It uses introspection to display the properties and events that are available within the Colors Bean

// Show properties and events.

```
import java.awt.*;
```

```
import java.beans.*;
```

```
public class IntrospectorDemo {
```

```
 public static void main(String args[]) {
```

```
 try {
```

```
 Class c = Class.forName("Colors");
```

```
 BeanInfo beanInfo = Introspector.getBeanInfo(c);
```

```
 System.out.println("Properties:");
```

```
 PropertyDescriptor propertyDescriptor[] =
```

```
 beanInfo.getPropertyDescriptors();
```

```
 for(int i = 0; i < propertyDescriptor.length; i++)
```

```
 {
```

```
 System.out.println("\t" + propertyDescriptor[i].getName());
```

```
 }
```

```
 System.out.println("Events:");
```

```
 EventSetDescriptor eventSetDescriptor[] =
```

```
 beanInfo.getEventSetDescriptors();
```

```
 for(int i = 0; i < eventSetDescriptor.length; i++)
```

```
 {
```

```
 System.out.println("\t" + eventSetDescriptor[i].getName());
```

```
 }
```

```
 }
```

```
 catch(Exception e)
```

```
 { System.out.println("Exception caught. " + e);
```

```
 } } } The output : Properties: rectangular Events:
```

```
mouseWheel mouse mouseMotion component
```

```
hierarchyBounds focus hierarchy propertyChange
```

```
inputMethod key
```

## 22. Explain the purpose and usage of the following classes in the context of Java Beans:

**Introspector, PropertyDescriptor, EventSetDescriptor and MethodDescriptor**

**Introspector:** The `Introspector` class provides several static methods that support introspection. Of most interest is `getBeanInfo()`.



- This method returns a **BeanInfo** object that can be used to obtain information about the Bean.
- The **getBeanInfo( )** method has several forms, including the one shown here:

*static BeanInfo getBeanInfo(Class bean) throws IntrospectionException*

- The returned object contains information about the Bean specified by bean

**PropertyDescriptor:** The *PropertyDescriptor* class describes a Bean property.

- It supports several methods that manage and describe properties.
- For example, you can determine if a property is bound by calling **isBound( )**.
- To determine if a property is constrained, call **isConstrained( )**.
- You can obtain the name of property by calling **getName( )**

**EventSetDescriptor:** The *EventSetDescriptor* class represents a Bean event.

- It supports several methods that obtain the methods that a Bean uses to add or remove event listeners, and to otherwise manage events.
- For example, to obtain the method used to add listeners, call **getAddListenerMethod( )**.
- To obtain the method used to remove listeners, call **getRemoveListenerMethod( )**.
- To obtain the type of a listener, call **getListenerType( )**.
- You can obtain the name of an event by calling **getName( )**.

**MethodDescriptor:** The *MethodDescriptor* class represents a Bean method.

- To obtain the name of the method, call **getName( )**.
- You can obtain information about the method by calling **getMethod( )**, shown here: *Method getMethod( )*
- An object of type *Method* that describes the method is returned

**23. Write a note on EventDescriptor and PropertyDescriptor class**

*Refer the 22 answer in long question*